

Jev-like Transformer: A Parallel Decision Model Based on Shared State and Dynamic Candidate Sets

Abstract

Traditional Large Language Models (LLMs) mainly use an autoregressive generation mechanism: given a context, they generate output token by token. When a task requires answering a large number of independent questions under the same context, this mechanism introduces a large amount of repeated computation.

This paper proposes a Jev-like Transformer architecture: the common context is represented as a shared State, while multiple questions to be answered are represented as parallel Question Tokens. A Transformer computes independent representations for each question in a single forward pass. Candidate Answer embeddings are then matched against the question representations to produce variable-length logits, followed by softmax to obtain a probability distribution for each question.

The key characteristics are: shared context computation, parallel question inference, and dynamically determined output spaces based on candidate answers, without requiring different-sized output Heads for different questions.

1. Core Idea

Traditional:

State
↓
Transformer
↓
token1 → token2 → token3 → ... → answer

Multiple:

State → Question 1 → Generate Answer
State → Question 2 → Generate Answer
State → Question 3 → Generate Answer
...

A large amount of computation is repeated.

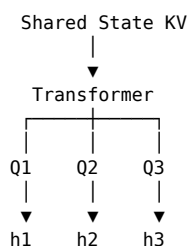
Proposed:

State → Backbone → $\begin{cases} \text{question1} \rightarrow \text{answer1} \\ \text{question2} \rightarrow \text{answer2} \\ \text{question3} \rightarrow \text{answer3} \end{cases}$

One shared State and multiple Questions enter the Transformer in parallel, and a single forward pass simultaneously obtains the answers for multiple questions.

2. Model Architecture

$S = (s_1, s_2, \dots, s_m)$
 $Q_i = (q_{i1}, q_{i2}, \dots, q_{ik_i})$
 $H_s = \text{Transformer}(S)$
 $h_i \in \mathbb{R}^d$



- State: the common context shared by all questions;
- Question: the query tokens belonging to each individual question;

- h_i : the semantic vector obtained after the i -th question passes through the Transformer.

3. Dynamic Candidate Head

Q1: Is a refund required? Answers {Yes, No}; hidden $\rightarrow$ Linear($d,2$) $\rightarrow$ logits[2]

Q2: What type of user? Answers {Normal User, High-Value User, Risk User, Blacklisted User}; hidden $\rightarrow$ Linear($d,4$) $\rightarrow$ logits[4]

Q3: 17 candidates $\rightarrow$ hidden $\rightarrow$ Linear($d,17$) $\rightarrow$ logits[17]

A fixed-size FFN Head cannot simply be used for every Question.

Otherwise, different numbers of answers require different output dimensions and therefore different output Heads.

4. Candidate Embedding + Shared Scorer

Instead of allowing the Head to determine the output dimension, let the Candidate Set determine the output dimension.

$e_a \in \mathbb{R}^d$

$h_i \in \mathbb{R}^d$

$z_{ij} = f(h_i, e_{ij})$

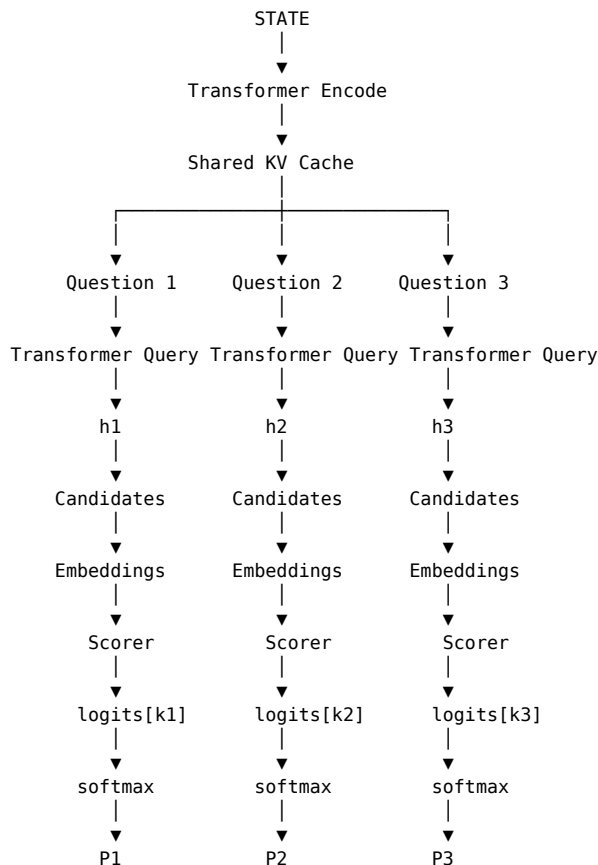
$z_{ij} = h_i \cdot e_{ij} / \sqrt{d}$

$P(a_{ij} | S, Q_i) = \text{softmax}_j(z_{ij})$

$h_i \rightarrow e(a1), e(a2), e(a3) \rightarrow \text{Shared Scorer} \rightarrow \text{logits}[N_i] \rightarrow \text{softmax} \rightarrow \text{probabilities}$

Examples: Q1 has 2 candidates, Q2 has 5 candidates, and Q3 has 100 candidates. The model itself does not need to change.

5. Complete Architecture



$P(a_{ij} | S, Q_i) = \text{softmax}_j(f(h_i, e_{ij}))$

- S : shared State
- Q_i : the i -th Question
- h_i : the representation of the i -th Question
- a_{ij} : the j -th candidate answer of the i -th Question
- e_{ij} : the embedding of the j -th candidate answer
- z_{ij} : the logit of the j -th candidate answer

6. Why It Is Important

Traditional LLM output space: $V \approx 10^4 \sim 10^5$

Jev-like output: $A_i = \{a_{i1}, a_{i2}, \dots, a_{ik_i}\}$

Each question has its own dynamic output space.

- Classification
- Routing
- Agent Decision
- Tool Selection
- Risk Assessment
- Intent Recognition
- Multi-condition Decision
- Ranking
- Structured Decision

Especially suitable for: one State + a large number of independent Questions + different numbers and different contents of candidate answers for each Question.

7. Engineering Implementation

7.1 The First Version Does Not Need to Modify the Transformer

Use an existing HuggingFace Transformer. Input State tokens + Question tokens. Construct an Attention Mask: Question → can see State; Question → can see itself; Question → does not need to see other Questions.

	State	Q1	Q2	Q3
State	✓	✓	✓	✓
Q1	✓	✓	×	×
Q2	✓	×	✓	×
Q3	✓	×	×	✓

Multiple Questions are computed in a single Transformer forward pass.

7.2 Simplest First Implementation

Organize State and Questions as [STATE TOKENS] [QUESTION TOKENS], and construct the corresponding Attention Mask. Q1, Q2, Q3 are processed in parallel within one batch.

8. PyTorch Scorer

```
class CandidateScorer(nn.Module):
    def forward(self, h, candidate_embeddings):
        # h: [B, D]
        # candidate_embeddings: [B, K, D]

        logits = torch.einsum(
            "bd,bkd->bk",
            h,
            candidate_embeddings
        )

        return logits
logits = scorer(h, candidates)

probs = torch.softmax(logits, dim=-1)
K can differ. Batch padding: Q1 K=2; Q2 K=5; Q3 K=17.

logits = logits.masked_fill(~candidate_mask, -inf)
```

9. More Powerful Scorer

```
z = f([h; e])
h/e concat → MLP → scalar
or
z = hT W e
logit = h @ W @ e
```

Another option is to use separate MLPs to transform the Question embedding and Candidate embedding, followed by a dot product.

10. Candidate Sources

A. Fixed Candidate Vocabulary. For example: ["yes", "no"] → embedding table.

B. Natural Language Candidates. For example, Question: “Which action should the customer take?”
Candidates: refund / exchange / reject / manual review. Encode the candidate text into embeddings and perform scoring.

C. Runtime Dynamic Candidates. For Agent tool selection: Question “Which tool should be called next?”
Candidates: search_web / database / calculator / send_email / browser. The Candidate Set can be generated dynamically at runtime. The model only needs to score these candidates.

11. Training

A training sample contains: State, Question, Candidates, Correct Answer.

Example: customer refund, yes/no, label yes.

$$L = -\log P(a^* | S, Q)$$

State S ; $Q_1 \rightarrow A_1, Q_2 \rightarrow A_2, \dots, Q_N \rightarrow A_N$

$$L = \sum_i L_i$$

Multiple independent decision problems are solved simultaneously under the same State.

12. Difference from Traditional LLMs

	Traditional LLM	Jev-like Model
Output unit	Token	Candidate
Output space	Fixed Vocabulary	Question-specific
Inference	Autoregressive	Parallel decision
Multiple questions	Repeated generation	Batch parallel
State	Participates in each generation	Shared KV
Output length	Variable token sequence	Number of candidates
Main task	Text generation	Decision / Classification / Selection

It is not simply “adding multiple classification heads to an LLM.” More accurately, it changes the output paradigm:

Token Generation → Candidate Scoring

13. Further Direction

State + Questions + Candidate Actions
State → Transformer → Tool Q / Action Q / Risk Q

↓
candidates
↓
score

Instead of generating an explanation, directly output probabilities:

```
search_web 0.71
database   0.18
calculator 0.07
browser    0.04
LLM
├── Reasoning / Generation
├── Decision Transformer
│   ├── Classification
│   ├── Routing
│   ├── Tool Selection
│   ├── Ranking
│   └── Agent Control
```

14. Minimum Viable System

V0: Qwen/Llama -> State + Question -> Transformer -> Question hidden -> Candidate embedding -> dot product -> softmax. First prove that the model can perform the task.

V1: Multiple Questions -> one batch forward -> validate throughput.

V2: State KV Cache -> multiple Questions share State KV.

V3: Runtime Dynamic Candidates.

V4: Inference optimization: Paged KV Cache, FlashAttention, Continuous Batching, Candidate Embedding Cache, Tensor Parallel, GPU fused scorer.

Ultimately, form a high-speed Decision Model designed for the Agent Decision Loop.

15. Core Contributions

- ① Shared State — same context only public calculation once.
- ② Parallel Questions — multiple Questions in the same Transformer inference.
- ③ Dynamic Candidate Set — each Question has its own Candidate Set, and the output dimension is dynamically determined by the Candidate Set.
- ④ Shared Candidate Scorer — no separate Prediction Head is required for different candidate counts; Candidate Embedding + Shared Scorer is used instead.

$S + \{Q_1, \dots, Q_n\} \rightarrow \{h_1, \dots, h_n\} \rightarrow \text{Candidate Scoring} \rightarrow \{P_1, \dots, P_n\}$

$P_i = \text{softmax}(f(h_i, E_i))$

$|E_1|, |E_2|, \dots, |E_n|$ can all be different.

16. Conclusion

The core is not simply multiple classification heads. It redefines the output paradigm of the Transformer:

Autoregressive Language Generation
->
Parallel Candidate-based Decision
Shared State
+
Parallel Questions
+
Dynamic Candidates
+
Shared Scorer

The key idea:

The Head no longer determines the output dimension; the Candidate Set determines the output dimension.

A unified Transformer can simultaneously process different numbers and different contents of candidates under the same State. This architecture can be applied to classification, routing, Tool Selection, Ranking, Risk Assessment, and Agent Action Selection.

From an engineering perspective, it can be built directly on an existing Qwen/Llama Transformer without redesigning the Transformer backbone. The first version only requires Attention Mask, Question Batch, Candidate Embedding, and Shared Scorer.